CS 483 Project 3
Responsive Smart Home Dashboard UI

Objective

In this project, you will be tasked with developing a responsive UI for a smart home dashboard using CSS Grid and Flexbox, along with semantic HTML. The goal is to closely replicate the provided design spec, which includes multiple widgets for controlling smart home devices. You will receive screenshots of the UI design to serve as a reference, and your objective is to develop the UI using HTML and CSS, without relying on any external CSS frameworks. The key is to apply responsive design principles, ensuring that the dashboard works well across different screen sizes.

Layout and Structure

You are required to create the layout using **CSS Grid** for the overall structure, with the header, footer, sidebar, and main content area clearly defined. The main content area will feature a grid layout, with the first column dedicated to a room picture, and the remaining columns will contain widgets for controlling different aspects of the smart home. Use **Flexbox** within the grid elements where necessary to align items or center content.

Ensure that you use **semantic HTML** elements to structure your page. Avoid using generic div elements unless necessary; instead, use appropriate tags such as section, header, footer, and article to describe the content and structure. This will help maintain accessibility and improve the semantic value of your code.

Responsive Design

The design should be responsive, meaning that it should adapt to various screen sizes, especially smaller screens like mobile devices. Use **media queries** to adjust the layout for a smaller screen. The layout should adjust such that the sidebar and main content stack appropriately on smaller screens, ensuring that the content remains easily readable and usable.

To achieve a responsive layout, avoid using fixed pixel values for widths, heights, or margins. Instead, use relative units like `rem` and `em` for font sizes and dimensions. In CSS, the rem unit (which stands for *root em*) is relative to the font size of this root element. By default, most browsers set the font size of the root element (<html>) to 16px. This means that 1rem is equal to 16px unless otherwise modified. When you use rem for sizing, it ensures that the layout and text scale consistently based on the root font size, making it easier to create scalable and accessible designs. If the root element's font size is adjusted (e.g., setting it to 10px or 18px), all rem units will adjust accordingly, providing a more flexible approach to layout and typography.

Design Elements

You task is to replicate the layout and styling based on the provided design. This includes the header, sidebar, and the main content area. The sidebar should include an image (representing the user's home exterior) and a list indicating placeholder for various application features and settings such as *history*, *gallery*, *rules*, a*nalytics*, etc. that a user could navigate to. The main content area will display a large image of the camera feed of a room, along with multiple widgets such as *temperature*, *light control*, *energy usage*, and *security*. A welcome message for the user should be placed in the top right corner.

Use HTML `input type="range"` to create the temperature slider inside the *Temperature* widget. You can define the `min, max,` and `value` (initial value) for it. The slider can be styled using CSS, and later you can use JavaScript (not for this project) to get the current temperature, and provide functionality to your UI.

I used a dark background, with lighter accent colors stacked on top in a complementary color palette. You are not required to match these colors. Feel free to choose your own palette. The key is to maintain aesthetic consistency throughout the layout. Also, prioritize usability by ensuring that the text is clear, legible, and accessible for a smooth user experience.

Here are the colors I used, in case you'd like to reference them:

- Overall container background: `black`
- Main content background: Dark grey (`#252E42`)
- Header, footer, widget: Slate grey (`#2c3e50`)
- Sidebar: Dark teal (`#1b4a5e`)
- Company Name/Logo (Smart Home): `lightskyblue`
- Text, button: `lightgoldenrodyellow, lightblue, lightgray,` etc.

Provided Assets

Use only the provided images for the project, as we have the appropriate permissions for these assets. However, you're welcome to source icons from other approved platforms if you prefer.

- Living room: `living-room.jpeg`
- House exterior: `home-front-door.jpeg`
- UI icons: `ui-icons.jpeg`

Extra Credit

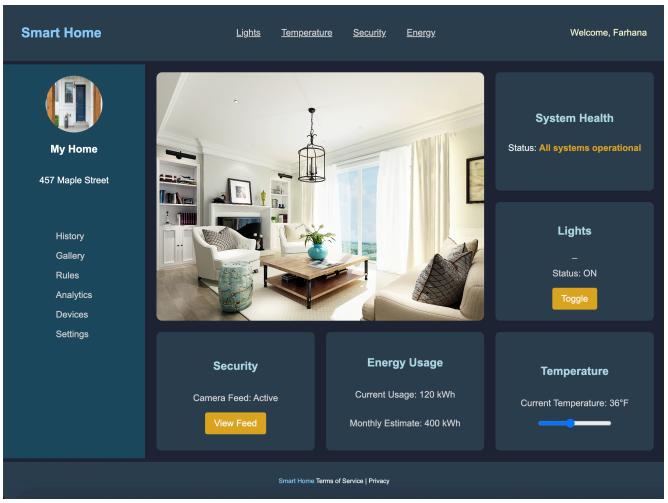
For extra credit, you can add *icons* to the sidebar and/or widgets. Additionally, consider adding a *text overlay* (e.g., Currently Streaming - Living Room - Jan 24, 2025, 10:35 AM) on top of the room image to indicate that this is a live video feed. Feel free to get creative and enhance user experience using a CSS skill without modifying the overall layout. Be sure to describe the extra credit work you've done in the **README**.

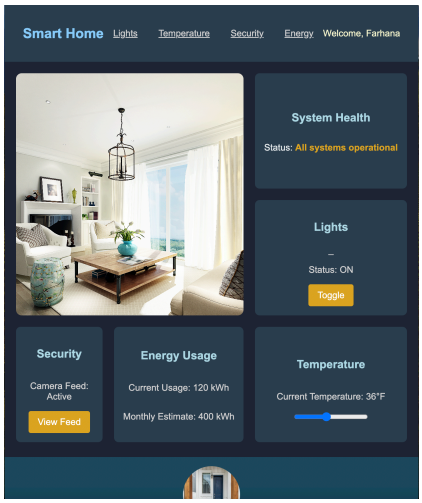
Key Points to Remember

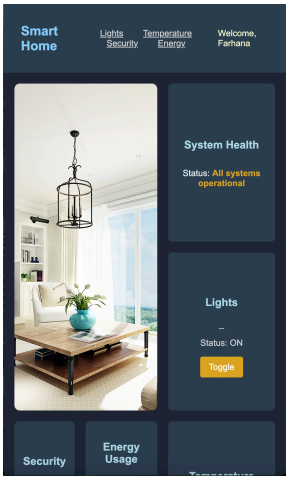
- Use semantic HTML elements such as <header>, <nav>, <main>, <section>, <footer>, and others to structure your page properly.
- Implement the layout using CSS Grid for the main content and Flexbox to align items inside a grid cell. You can nest grid elements; I did for this design.
- Hover effects are required for all buttons and links in the layout. Ensure that buttons and links provide interactive feedback when hovered over.
- Ensure the layout adapts properly to a smaller screen size by using media queries. Avoid using fixed pixel values for dimensions as much as possible; I used a fixed height/width for the small image only.
- Make sure to handle the images properly, ensuring they are appropriately sized and correctly displayed within the layout without causing any overflow or layout issues.

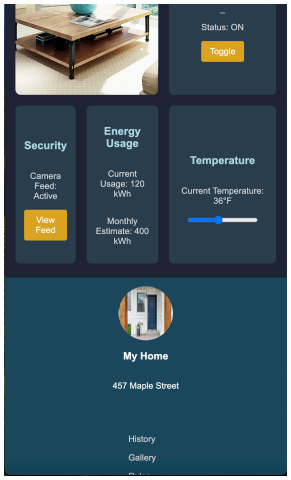
Here are some screenshots showcasing the layout changes for different screen widths, with a noticeable shift in design for smaller screens.

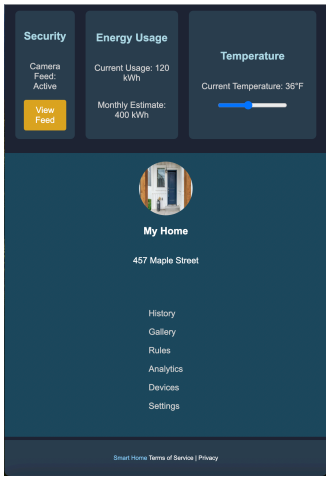
Enjoy bringing your smart home dashboard to life!











Score Breakdown

Criteria	Point %
Proper use of CSS Grid for layout structure	40%
Effective use of Flexbox to align elements	20%
Use of media queries and rem units for responsiveness	15%
HTML and CSS best practices - semantic HTML, clean readable CSS (Document your CSS code)	10%
Functionality - hover/click style changes for links and buttons	5%
Visual fidelity to the design spec	10%
Bonus: Incorporate icons (5%), Create a text overlay for image (5%), or Some other user experience enhancement using CSS (5%)	up to 15%

Submitting your Solution

Organize your HTML file and any supporting files into the correct directory structure, ensuring that the grader can render the HTML without any extra steps. The main folder should contain the `index.html` file and two additional folders: `styles` for your CSS file and `assets` for your images. Once organized, compress the files into a ZIP archive and name it as follows: ***<first name>_<last name>*_proj3.zip**

Your project archival should include a text file named **README** with the following information:

- Your **name** and **email address**
- A **list of all the files** in your archive with a brief one-line description of each file
- Write a paragraph for each **Extra Credit** work you completed for the project, describing the specific CSS techniques you used to implement these enhancements

Note that:

- Failure to adhere to the directory structure specifications, omitting the README file, or requiring additional communication from the grader to view or test your HTML page will result in point deductions.
- **Before finalizing your submission**, I strongly recommend extracting your archive into a new directory and testing that all files are included and the HTML renders correctly.